Change Request: integrazione Docker Compose in `docker-dev`

1. Obiettivo

Integrare Docker Compose in `docker-dev` per gestire, oltre al container di sviluppo, servizi opzionali del progetto — ad esempio PostgreSQL, Redis, Mailpit o MinIO — mantenendo:

- il container di sviluppo effimero di default;
- i volumi di progetto persistenti, come `venv.<container_name>`;
- la pipeline a script numerati in `langs/<TYPE>/bin/`;
- la futura migrazione di `docker-dev.sh` da Bash a Perl.

La soluzione scelta è **Perl**, con `YAML::XS` per il parsing dei manifest e **Template::Toolkit** per il rendering dei template Compose. I template per servizio avranno estensione `.yaml.tt`, non `.yaml.j2`, perché `.j2` è la convenzione Jinja2.

2. Stato attuale

2.1 Flusso attuale

`docker-dev.sh` esegue questi passaggi principali:

1. Valida `ORG`, `BASE`, Docker e il target del progetto.
2. Determina `P_TYPE`, `P_TARGET` e `P_NAME` dal percorso, ad esempio `Python/foo`.
3. Calcola:
   - `IMAGE_NAME`, default `${P_TARGET}.${P_TYPE}`, normalizzato in minuscolo;
   - `CONTAINER_NAME`, default `${IMAGE_NAME}.${P_NAME}`.
4. Invoca `docker-run-extra-args.sh`, che risolve `DOCKER_RUN_EXTRA_ARGS` da ambiente, file di progetto oppure fallback `--rm -it`.
5. Applica i template con `apply-templates.sh`.
6. Esegue, in ordine, gli script numerati presenti in `langs/<TYPE>/bin/`.

Per Python, la pipeline attuale è:

Script	Funzione
`10.setup-volumes.sh`	Crea e inizializza `venv.<container_name>` e `uv-python`, impostando ownership `UID:GID`.
`20.build.sh`	Costruisce `${IMAGE_NAME}` dal `Dockerfile` del progetto, se non esiste.

Script	Funzione
`30.run.sh`	Rimuove un eventuale container omonimo e avvia il container con bind mount del progetto, volume `venv` e volume `uv-python`.

2.2 Comportamento di runtime attuale

`30.run.sh` usa sempre `docker run`; `DOCKER_RUN_EXTRA_ARGS` contiene di default `--rm -it`, quindi il container di sviluppo è effimero. I mount attuali sono:

```
${PROJECT_DIR}:/app
${VOLUME_NAME}:/app/.venv
uv-python:/home/${USER_}/.local/share/uv/python
```

dove `VOLUME_NAME` è `venv.${CONTAINER_NAME}`, salvo override tramite ambiente.

2.3 Limiti attuali

- Non esiste un modello dichiarativo per servizi aggiuntivi.
- Ogni servizio dipendente dovrebbe essere avviato manualmente con comandi `docker run` separati.
- Non ci sono reti Compose, healthcheck, dipendenze ordinate o profili.
- `docker-run-extra-args.sh` produce opzioni per `docker run`, non per `docker compose`.
- La logica di naming è distribuita tra `docker-dev.sh` e gli script della pipeline.

3. Soluzione proposta

3.1 Principi

1. `docker-dev.sh` resta il front-end e l'orchestratore.
2. `langs/<TYPE>/` definisce l'ambiente di sviluppo.
3. Una nuova cartella `services/<TYPE>/` definisce i servizi riutilizzabili.
4. `40.services.sh` è un **resolver + generatore** neutrale: non contiene logica specifica di PostgreSQL, Redis o altri servizi.
5. I servizi sono descritti da un manifest `service.yaml` e da template `compose.yaml.tt`.
6. Il container `dev` resta effimero; i volumi dichiarati restano persistenti.
7. Compose è il backend dichiarativo per reti, dipendenze, healthcheck e servizi.

3.2 Layout proposto

```
docker-dev/
├── docker-dev.sh
├── apply-templates.sh
├── docker-run-extra-args.sh
```

```
├── bin/
│   └── docker-dev-services.pl
├── langs/
│   └── Python/
│       ├── templates/
│       │   ├── Dockerfile
│       │   ├── docker-run-extra-args
│       │   ├── entrypoint.Python.sh
│       │   └── .docker_container/
│       └── bin/
│           ├── 10.setup-volumes.sh
│           ├── 20.build.sh
│           ├── 30.run.sh
│           └── 40.services.sh
└── services/
    ├── postgres/
    │   ├── service.yaml
    │   ├── templates/
    │   │   ├── compose.yaml.tt
    │   │   ├── Dockerfile
    │   │   └── entrypoint.postgres.sh
    │   └── bin/
    │       ├── 10.setup-volumes.sh
    │       └── 20.build.sh
    ├── redis/
    │   ├── service.yaml
    │   ├── templates/
    │   │   └── compose.yaml.tt
    │   └── bin/
    └── mailpit/
        ├── service.yaml
        ├── templates/
        │   └── compose.yaml.tt
        └── bin/
```

4. Interfaccia utente

4.1 Comandi previsti

```
# Sviluppo senza servizi aggiuntivi: comportamento attuale
docker-dev --target=Python/foo

# Servizi tramite alias logici
docker-dev --target=Python/foo --with=db,cache

# Servizi tramite tipi espliciti
docker-dev --target=Python/foo --with=postgres,redis

# Parametri per istanza
docker-dev --target=Python/foo \
  --with=db:tag=16,host_port=15432 \
  --with=cache:tag=7

# Build immagini servizio
```

```
docker-dev --build-service=postgres,redis

# Ispezione del Compose merged
docker-dev --target=Python/foo --with=db,cache --config

# Arresto stack
docker-dev --target=Python/foo --down
```

4.2 Alias logici

Alias CLI	Tipo servizio	Nome logico Compose
`db`	`postgres`	`db`
`cache`	`redis`	`cache`
`mail`	`mailpit`	`mail`
`object`	`minio`	`object`

L'alias è l'interfaccia stabile per l'utente; il tipo identifica l'implementazione in `services/`. Così un progetto può in futuro sostituire `db` con MySQL senza cambiare il modo in cui lo richiama.

5. Naming e convenzioni

5.1 Naming attuale da preservare

```
IMAGE_NAME     = dev.python
CONTAINER_NAME = dev.python.foo
VOLUME_NAME    = venv.dev.python.foo
```

Nel codice attuale, `IMAGE_NAME` è derivato da `${P_TARGET}.${P_TYPE}` e `CONTAINER_NAME` da `${IMAGE_NAME}.${P_NAME}`. Il volume Python è `venv.${CONTAINER_NAME}`.

5.2 Naming proposto per i servizi

```
IMAGE_SERVICE  = dev.postgres:16
IMAGE_SERVICE  = dev.redis:7

VOLUME_SERVICE = data.dev.postgres.foo
VOLUME_SERVICE = data.dev.redis.foo
```

Regole:

- Il nome del volume include il **tipo di servizio**, non l'alias logico: `data.dev.postgres.foo`, non `data.dev.db.foo`.
- Il nome del volume include il progetto: `.foo`.
- I volumi dei servizi possono essere gestiti da Compose oppure dichiarati `external: true` se richiedono inizializzazione/ownership dedicata.

- Il volume `venv.dev.python.foo` resta gestito da `langs/Python/bin/10.setup-volumes.sh`.

6. Manifest del servizio

Ogni servizio è descritto da `services/<type>/service.yaml`.

6.1 Esempio PostgreSQL

```
name: postgres
image: dev.postgres
default_tag: "16"
profiles:
  - db
container_port: 5432
default_host_port: 15432
data_volume:
  name: "data.dev.postgres.{project}"
  mount: /var/lib/postgresql/data
  external: true
environment:
  POSTGRES_USER: app
  POSTGRES_PASSWORD: app
  POSTGRES_DB: app
connection_env:
  DATABASE_URL: "postgres://app:app@{service}:5432/app"
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U app -d app"]
  interval: 5s
  timeout: 3s
  retries: 10
```

6.2 Esempio Redis

```
name: redis
image: dev.redis
default_tag: "7"
profiles:
  - cache
container_port: 6379
default_host_port: 16379
data_volume: null
environment:
  REDIS_ARGS: "--save '' --appendonly no"
connection_env:
  REDIS_URL: "redis://{service}:6379/0"
healthcheck: null
```

6.3 Contratto del manifest

Campi obbligatori:

- `name`
- `image`
- `profiles`
- `container_port`
- `connection_env`

Campi opzionali:

- `default_tag`
- `default_host_port`
- `data_volume`
- `environment`
- `healthcheck`
- `build`
- `volumes`
- `networks`
- `depends_on`

7. Template Compose per servizio

Ogni servizio ha un template Template::Toolkit:

```
services/postgres/templates/compose.yaml.tt
```

Esempio:

```
services:
  [% logical_name %]:
    image: [% image %]
    profiles:
      - [% profile %]
    restart: unless-stopped
[% IF host_port %]
    ports:
      - "127.0.0.1:[% host_port %]:[% container_port %]"
[% END %]
[% IF data_volume %]
    volumes:
      - [% data_volume %]:[% data_mount %]
[% END %]
    environment:
[% FOREACH pair IN environment.pairs %]
      [% pair.key %]: [% pair.value %]
```

```
[% END %]
[% IF healthcheck %]
    healthcheck:
      test: [% healthcheck_test %]
      interval: [% healthcheck_interval %]
      timeout: [% healthcheck_timeout %]
      retries: [% healthcheck_retries %]
[% END %]
```

Il template riceve solo valori già risolti e validati dal resolver Perl:

```
logical_name
image
profile
host_port
container_port
data_volume
data_mount
environment
healthcheck_test
healthcheck_interval
healthcheck_timeout
healthcheck_retries
```

8. File generati

8.1 Posizione

```
<PROJECT_DIR>/.docker-dev/
├── compose.yaml
├── compose.user.yaml
├── .env
└── generated/
    ├── compose.services.yaml
    ├── profiles
    └── services.lock
```

8.2 `compose.yaml`

Generato da `apply-templates.sh` a partire da `langs/Python/templates/compose.yaml.tt`. Contiene il servizio `dev`, il mount del progetto, i volumi di linguaggio e l'entrypoint.

8.3 `compose.user.yaml`

File facoltativo, modificabile dall'utente o dal progetto. Non viene mai sovrascritto da `docker-dev`.

8.4 `generated/compose.services.yaml`

Generato da `40.services.sh` / `bin/docker-dev-services.pl` prima di ogni esecuzione Compose. Contiene solo i servizi richiesti nella riga di comando, le relative dipendenze dal servizio `dev` e i volumi dei servizi.

8.5 `generated/profiles`

File di testo con un profilo per riga:

```
db
cache
```

8.6 `generated/services.lock`

File opzionale ma consigliato, contenente almeno:

```
{
  "project": "dev.python.foo",
  "services": ["postgres", "redis"],
  "images": {
    "db": "dev.postgres:16",
    "cache": "dev.redis:7"
  },
  "manifest_hash": "..."
}
```

Serve a rilevare cambiamenti di manifest, parametri o versione template.

9. Comportamento del resolver

`40.services.sh` deve:

1. Ricevere `--with=db,cache` oppure `--with=postgres,redis`.
2. Risolvere ogni alias nel tipo di servizio.
3. Caricare `services/<type>/service.yaml` con `YAML::XS`.
4. Validare il manifest e i parametri CLI.
5. Calcolare immagine, tag, porta host, volume e variabili di connessione.
6. Renderizzare `services/<type>/templates/compose.yaml.tt`.
7. Aggregare i frammenti in `generated/compose.services.yaml`.
8. Aggiungere al servizio `dev` solo `environment` e `depends_on`.
9. Scrivere i profili in `generated/profiles`.
10. Validare il risultato con `docker compose config --quiet`.

Non deve:

- conoscere la semantica di PostgreSQL o Redis;
- contenere comandi `docker run` specifici di servizio;
- modificare `compose.user.yaml`;
- sovrascrivere il Compose base generato da `apply-templates.sh`;
- modificare `entrypoint`, `command`, `user` o `working_dir` del servizio `dev`.

10. Integrazione con la pipeline esistente

10.1 Ordine di esecuzione

```
docker-dev --target=Python/foo --with=db,cache

1. docker-dev.sh valida target, Docker, ORG/BASE
2. docker-dev.sh calcola IMAGE_NAME e CONTAINER_NAME
3. docker-run-extra-args.sh risolve argomenti legacy
4. apply-templates.sh genera .docker-dev/compose.yaml
5. langs/Python/bin/10.setup-volumes.sh crea venv.dev.python.foo e uv-python
6. langs/Python/bin/20.build.sh costruisce dev.python, se necessario
7. langs/Python/bin/40.services.sh:
   - risolve db -> postgres
   - risolve cache -> redis
   - genera .docker-dev/generated/compose.services.yaml
   - genera .docker-dev/generated/profiles
8. docker-dev.sh esegue docker compose config --quiet
9. docker-dev.sh esegue docker compose up -d
10. docker-dev.sh avvia la shell/task nel servizio dev
```

10.2 Modifiche a `docker-dev.sh`

Aggiungere il parsing di:

```
--with=...
--build-service=...
--config
--down
--dry-run
```

Variabili interne proposte:

```
DDEV_WITH=""
DDEV_BUILD_SERVICE=""
DDEV_COMPOSE_CONFIG_ONLY=false
DDEV_COMPOSE_DOWN=false
DDEV_DRY_RUN=false
DDEV_BUILD_MISSING=false
```

Il flusso Compose deve essere attivato solo se:

```
[[ -n "$DDEV_WITH" || -n "$DDEV_BUILD_SERVICE" || "$DDEV_COMPOSE_DOWN" == true ]]
```

In assenza di `--with`, `docker-dev` mantiene il comportamento attuale basato su `docker run`, garantendo compatibilità all'indietro.

10.3 Modifiche a `30.run.sh`

`30.run.sh` deve diventare un dispatcher:

- **Modalità legacy**: nessun `--with`; continua a usare `docker run --rm -it`.
- **Modalità Compose**: `--with` presente; usa `docker compose run --rm dev ...` oppure `docker compose exec dev ...`, secondo la modalità scelta.

Per la prima implementazione:

```
docker compose \
  -p dev.python.foo \
  -f .docker-dev/compose.yaml \
  -f .docker-dev/compose.user.yaml \
  -f .docker-dev/generated/compose.services.yaml \
  --profile db \
  --profile cache \
  run --rm dev
```

`run --rm` preserva il comportamento effimero attuale.

11. Scelte tecniche Perl

11.1 Dipendenze

```
cpanm YAML::XS Template
```

Oppure, dove disponibili nei repository:

```
sudo apt install libyaml-libyaml-perl libtemplate-perl
```

11.2 Parsing YAML

```
use YAML::XS qw(LoadFile);

my $manifest = LoadFile($manifest_path);
```

Non abilitare:

```
local $YAML::XS::LoadBlessed = 1;
```

Con `YAML::XS` aggiornato, `LoadBlessed` è disabilitato per default; questo è l'equivalente pratico della politica sicura di `yaml.safe_load` in PyYAML.

11.3 Templating

```
use Template;

my $tt = Template->new(
    ABSOLUTE => 1,
    STRICT   => 1,
);

my $output;
$tt->process($template_path, \%vars, \$output)
    or die $tt->error;
```

`STRICT => 1` è consigliato: una variabile non definita deve causare errore invece di produrre YAML incompleto.

12. Compatibilità e migrazione

12.1 Compatibilità all'indietro

Caso	Comportamento
`docker-dev --target=Python/foo`	Invariato: pipeline Python e `docker run --rm -it`
`docker-dev --target=Python/foo comando`	Invariato
`docker-dev --target=Python/foo --with=...`	Nuovo percorso Compose
`docker-dev --build-service=...`	Nuovo comando di build immagini servizio
Progetti esistenti senza `.docker-dev/`	Nessuna migrazione obbligatoria

12.2 Migrazione graduale

1. Aggiungere `40.services.sh` e `services/` senza modificare il default.
2. Introdurre `--with` come opt-in.
3. Generare `compose.yaml` solo quando viene richiesto Compose.
4. Mantenere `30.run.sh` dual-mode.
5. Dopo la stabilizzazione, valutare se rendere Compose il default per i nuovi progetti.

13. Test richiesti

13.1 Test unitari

- Parsing di `--with=db,cache`.
- Parsing di `--with=postgres,redis`.
- Parsing di parametri: `db:tag=16,host_port=15432`.
- Alias sconosciuti.
- Manifest mancante o YAML non valido.
- Template mancante.
- Variabile TT non definita.
- Conflitto di porta host.
- Nome volume con `{project}` non risolto.

13.2 Test di integrazione

- `docker-dev --target=Python/foo` continua a funzionare senza Compose.
- `docker-dev --target=Python/foo --with=db,cache` genera Compose valido.
- `docker compose config` produce la configurazione attesa.
- `docker compose up -d` avvia `dev`, `db` e `cache`.
- Il servizio `dev` riceve `DATABASE_URL` e `REDIS_URL`.
- `depends_on` usa `service_healthy` per PostgreSQL.
- `docker compose down` non rimuove `venv.dev.python.foo`.
- `docker compose down -v` non rimuove volumi esterni.
- Una seconda esecuzione riusa immagine e volumi esistenti.

13.3 Test di regressione

- Container name, image name e volume name invariati.
- Mount `/app`, `/app/.venv` e `uv-python` invariati nella modalità legacy.
- `DOCKER_RUN_EXTRA_ARGS` continua a funzionare in modalità legacy.
- `apply-templates.sh` non sovrascrive file utente senza `-o`.

14. Rischi e mitigazioni

Rischio	Impatto	Mitigazione
Merge Compose imprevisto	Servizio `dev` con configurazione sovrascritta	Il file servizi modifica `dev` solo per `environment` e `depends_on`
Porte host già occupate	`up` fallisce	Validazione preliminare e porta configurabile
YAML non valido	Stack non avviabile	`docker compose config --quiet` prima di `up`

Rischio	Impatto	Mitigazione
Dipendenze Perl mancanti	Script non eseguibile	Controllo all'avvio e messaggio di installazione
Volumi cancellati per errore	Perdita di `venv` o dati	Volumi di linguaggio `external: true`; `down -v` solo con conferma
Deriva tra manifest e template	Compose non coerente	Hash in `services.lock` e validazione schema
Comportamento diverso legacy/Compose	Confusione utente	Documentazione e messaggi espliciti di modalità

15. Criteri di accettazione

La change request è considerata completata quando:

1. `docker-dev --target=Python/foo` mantiene il comportamento attuale.
2. `docker-dev --target=Python/foo --with=db,cache` genera ed esegue uno stack Compose valido.
3. `40.services.sh` non contiene logica specifica di PostgreSQL, Redis o Mailpit.
4. Ogni servizio è definito in `services/<type>/service.yaml` e `services/<type>/templates/compose.yaml.tt`.
5. `venv.dev.python.foo` sopravvive a `docker compose down`.
6. Il container `dev` è effimero in modalità Compose.
7. `docker compose config --quiet` è eseguito prima di ogni `up`.
8. I test di regressione della pipeline Python passano.
9. La struttura e le convenzioni sono pronte per una futura riscrittura di `docker-dev.sh` in Perl.